

Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Informática

Escuela Profesional: Ing. Informática y de Sistemas

COMPUTACION GRÁFICA I

Práctica N° 06

TRANSFORMACIÓN 2D — TRASLACIÓN



Estudiante: Efrain Vitorino Marin

Código de estudiante: 160337

Profesor: Dr. Hans Harley Ccacyahuillca Bejar

Fecha: 11 de mayo de 2026

1. Introducción

El presente informe describe la implementación de una transformación geométrica de traslación en 2D usando C++ y OpenGL moderno. La figura utilizada en la escena es una letra E construida en la CPU mediante un arreglo de vértices, enviada a la GPU por medio de un VBO y transformada en el *vertex shader* con una matriz homogénea de orden 3×3 .

Aunque la carátula fue solicitada con el formato institucional indicado, el contenido técnico del documento corresponde al repositorio de la práctica de traslación 2D disponible en:

<https://github.com/lovito99/labografica/tree/main/Translate2D>

2. Objetivo

Implementar una traslación bidimensional en el pipeline programable de OpenGL, representando una figura con forma de letra E, aplicando la transformación mediante shaders y verificando visualmente el desplazamiento solicitado.

3. Fundamento teórico

En coordenadas cartesianas, una traslación mueve cada punto de una figura una cantidad fija en los ejes x e y . Si un punto inicial es $P = (x, y)$ y el vector de traslación es $\mathbf{t} = (t_x, t_y)$, entonces su nueva posición es:

$$P' = (x + t_x, y + t_y) \quad (1)$$

Para trabajar esta operación dentro del pipeline gráfico se emplean coordenadas homogéneas. La traslación en 2D se expresa con la matriz:

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

La posición transformada se obtiene mediante:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} \quad (3)$$

En esta implementación se utilizó el vector de traslación $(t_x, t_y) = (0.4, -0.3)$.

4. Descripción de la implementación

La aplicación se desarrolló en C++ usando GLFW para crear la ventana, GLEW para cargar funciones de OpenGL y shaders GLSL para ejecutar la transformación en la GPU. La letra E se modeló como la unión de cuatro rectángulos: una barra vertical y tres barras horizontales. Cada rectángulo fue descompuesto en dos triángulos, obteniendo un total de 24 vértices.

4.1. Definición de la geometría

Los vértices de la letra E se definen en la CPU mediante un arreglo lineal de tipo `float`. Este arreglo contiene las coordenadas normalizadas que luego son copiadas al VBO.

```
1 float vertices[] = {
2     // Barra vertical izquierda
3     -0.80f,  0.55f,  -0.60f,  0.55f,  -0.60f,  -0.55f,
4     -0.80f,  0.55f,  -0.60f,  -0.55f,  -0.80f,  -0.55f,
5
6     // Barra superior
7     -0.60f,  0.55f,  -0.10f,  0.55f,  -0.10f,  0.35f,
8     -0.60f,  0.55f,  -0.10f,  0.35f,  -0.60f,  0.35f,
9
10    // Barra central
11    -0.60f,  0.10f,  -0.28f,  0.10f,  -0.28f,  -0.10f,
12    -0.60f,  0.10f,  -0.28f,  -0.10f,  -0.60f,  -0.10f,
13
14    // Barra inferior
15    -0.60f, -0.35f,  -0.10f, -0.35f,  -0.10f, -0.55f,
16    -0.60f, -0.35f,  -0.10f, -0.55f,  -0.60f, -0.55f
17 };
```

Listing 1: Fragmento de definición de vértices de la letra E

4.2. Carga de datos al VBO

Una vez definidos los vértices, la geometría se envía a la GPU mediante un VAO y un VBO. El atributo de posición se asocia al `location = 0` del *vertex shader*.

```
1 glGenVertexArrays(1, &vao);
2 glGenBuffers(1, &vbo);
3
4 glBindVertexArray(vao);
5 glBindBuffer(GL_ARRAY_BUFFER, vbo);
6 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
7             GL_STATIC_DRAW);
8
9 glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(float),
10                      (void*)0);
11 glEnableVertexAttribArray(0);
```

```
10 glBindVertexArray(0);
```

Listing 2: Configuración del VAO y VBO

4.3. Aplicación de la traslación

La matriz de traslación se construye en la CPU y se envía como un *uniform* de tipo `mat3`. Posteriormente, el *vertex shader* multiplica esa matriz por cada vértice expresado en coordenadas homogéneas.

```
1 void matrizTraslacion(float tx, float ty, float M[9])
2 {
3     M[0] = 1; M[3] = 0; M[6] = tx;
4     M[1] = 0; M[4] = 1; M[7] = ty;
5     M[2] = 0; M[5] = 0; M[8] = 1;
6 }
```

Listing 3: Construcción de la matriz de traslación en C++

```
1 #version 330 core
2 layout(location = 0) in vec2 aPos;
3 uniform mat3 uTranslation;
4
5 void main() {
6     vec3 p = uTranslation * vec3(aPos, 1.0);
7     gl_Position = vec4(p.xy, 0.0, 1.0);
8 }
```

Listing 4: Vertex shader que aplica la traslación

Durante el renderizado, la traslación usada en la escena es la solicitada por la práctica:

```
1 float M[9];
2 matrizTraslacion(0.4f, -0.3f, M);
3 glUniformMatrix3fv(locMat, 1, GL_FALSE, M);
4 glDrawArrays(GL_TRIANGLES, 0, kVertexCount);
```

Listing 5: Aplicación de la traslación en el ciclo de renderizado

5. Resultado obtenido

La Figura 1 muestra el resultado de la ejecución del programa. Se observa la letra E renderizada y desplazada respecto a su posición original de definición, cumpliendo con la traslación especificada.

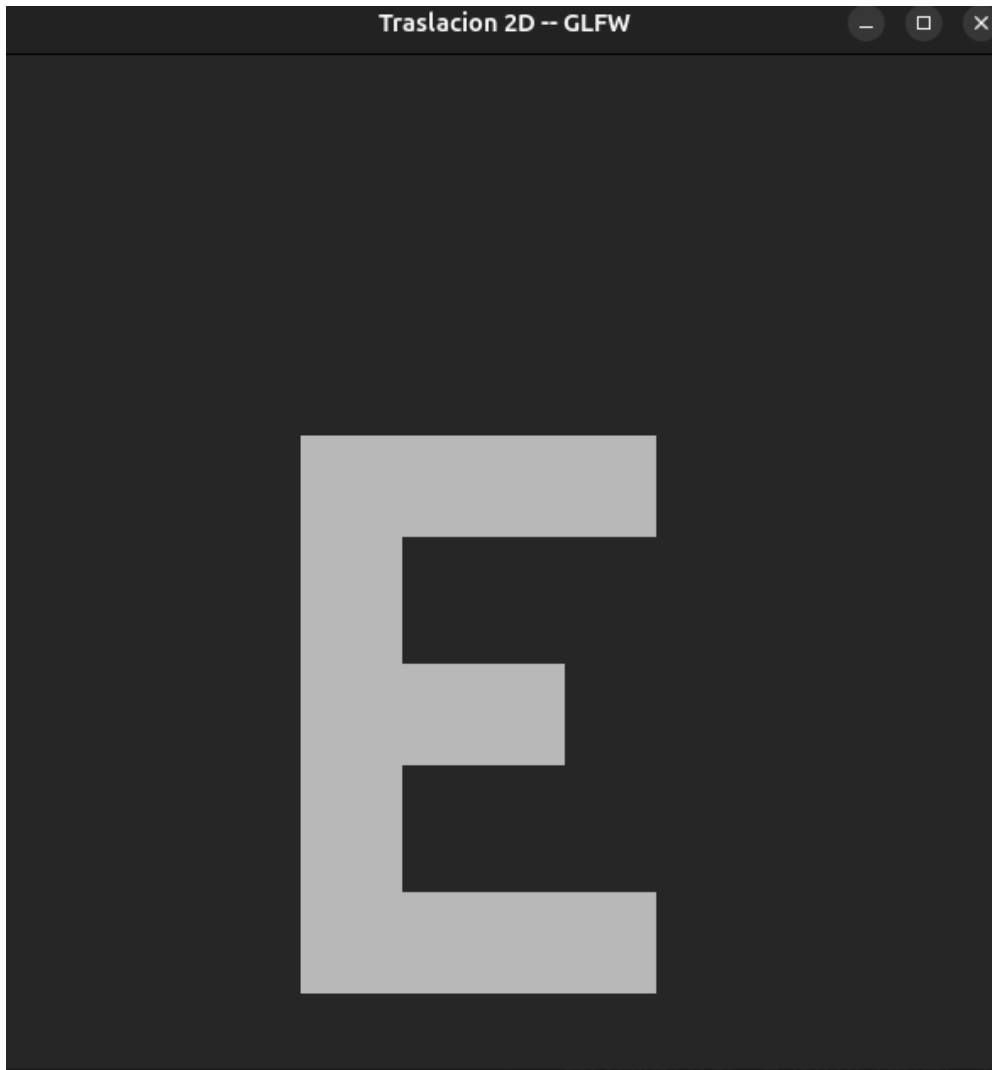


Figura 1: Resultado de la implementación de la traslación 2D de la letra E.

6. Conclusiones

La implementación desarrollada demuestra que una traslación 2D puede integrarse de forma clara dentro del pipeline moderno de OpenGL mediante shaders. La definición de la geometría en la CPU y su posterior procesamiento en la GPU permite separar la construcción de la figura del proceso de transformación.

El uso de coordenadas homogéneas y matrices de 3×3 facilita la representación matemática de la traslación. Además, la figura en forma de letra E evidencia visualmente el desplazamiento aplicado con los parámetros requeridos, confirmando el correcto funcionamiento de la implementación.